

Team Member Full Name	NetID
Chau Ta	cta
Brandon Martinez	bmarti32

Persistent Storage Design

We are using a SQLite database to persist our data. Our database includes the tables shown in Figure 1, which contains 4 tables. The main table is the User, which is Django's built-in user model.

Django's built-in User model (`django.contrib.auth`) uses a default User model. It is a primary-key-based model, which has a generated ID. The username, password, email, and optional name fields are standard data attributes for this model by default. The User is used as a foreign key for UserPlayBank and GameSession tables. Each User can have multiple GameSession but only one UserPlayBank.

UserPlayBank keeps track of the user's usage limit based on `user_id` and date that it was last reset. If a user does not exist yet, starting a game will create one for them with 3 play remainings and last reset date to be the date the new user starts the game. If a user already exists and wants to start a play, the server will reset their play remains based on the last time it was reset and the current date they are playing. Otherwise, if it is the same day, then the play remaining will be decremented once they start a game.

Each User can have multiple GameSession, which track the language, target word, status (won, lost, or active), number of attempts used, and the timestamps they were created and completed. Each GameSession can have multiple Guesses. These guesses are used to be loaded back in when a game is unfinished.

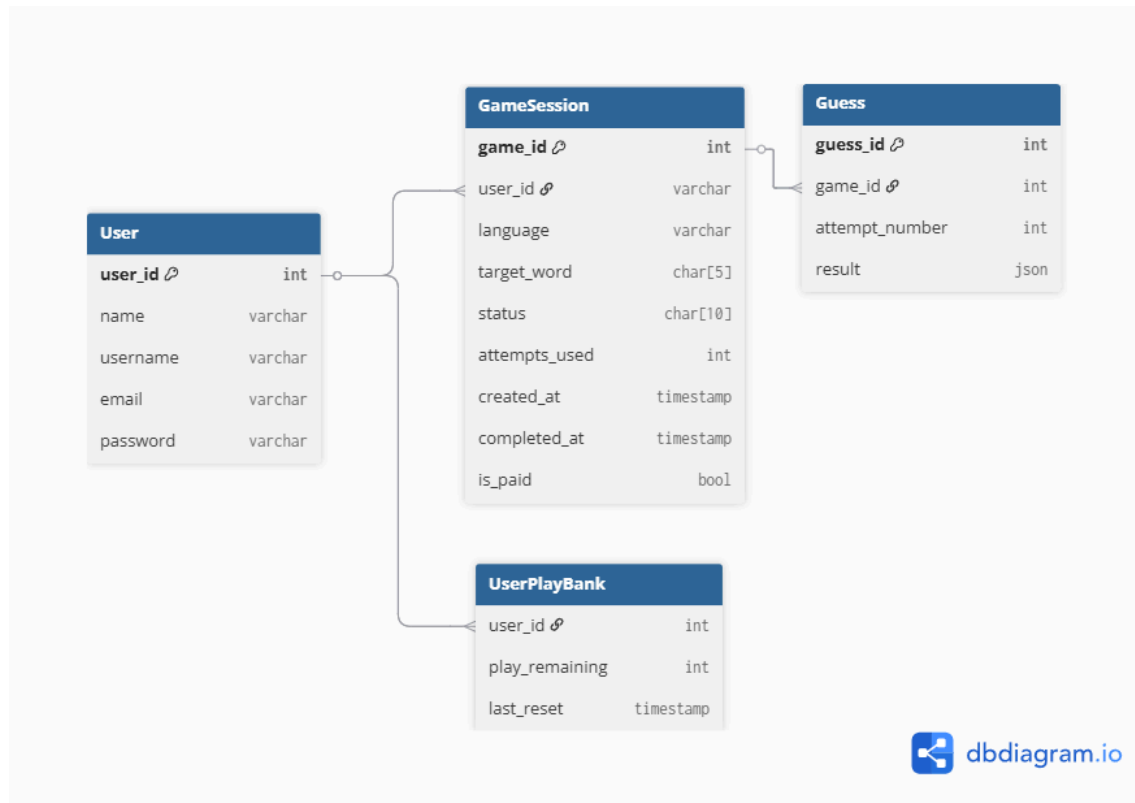


Figure 1 database schema

Demonstration of the Features

Feature 1: Sign Up / Sign in

Figure 2.1 shows the screenshot for creating a new User profile feature. Users can create a new account by providing their username, email, password, and optional name. The account creation account is accessed through the main login page if the user does not have an account yet.

Name (optional)

Username

Email

Password

CREATE ACCOUNT

ALREADY HAVE AN ACCOUNT? LOG IN

Figure 2.1 Screenshot for feature 1.1 showing signup page

Figure 2.2 shows the screenshot for the log-in page. Users can log in with their profile by providing their username and password. Users will be directed to this page if they are not logged in. In addition, if any users have not been authorized, any access to the rest of the website will be directed to this page using a built-in decorator from the Django authentication system.

Username

Password

LOG IN

CREATE ACCOUNT

Figure 2.2 Screenshot for feature 1.2 showing login page

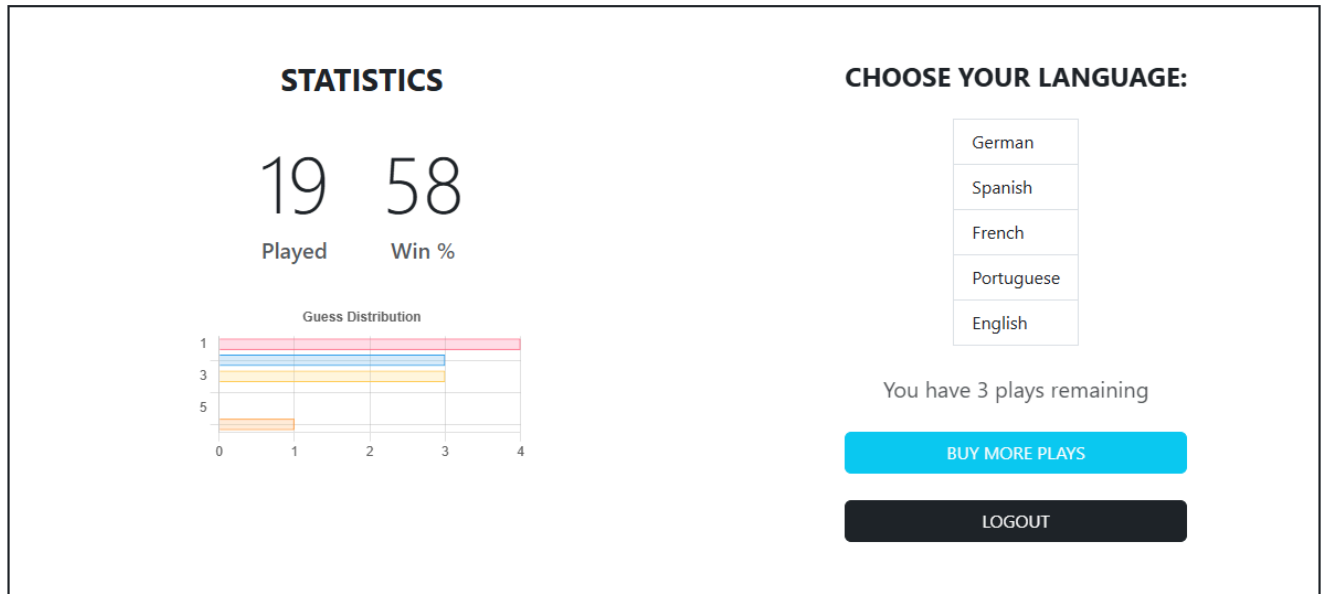


Figure 2.3 Screenshot for showing feature 1.3 showing logout button inside the dashboard

Feature 2: Gameplay

After logging in, the player can choose to start a game via the language selection in the dashboard, shown in Figure 2.3. After the user selects a language, a word is randomly chosen, and the user is redirected to the main game page shown in Figure 3.1. The game validates inputs and provides visual feedback to each guess. After the game, users can select Play Again, which redirects them back to the dashboard.

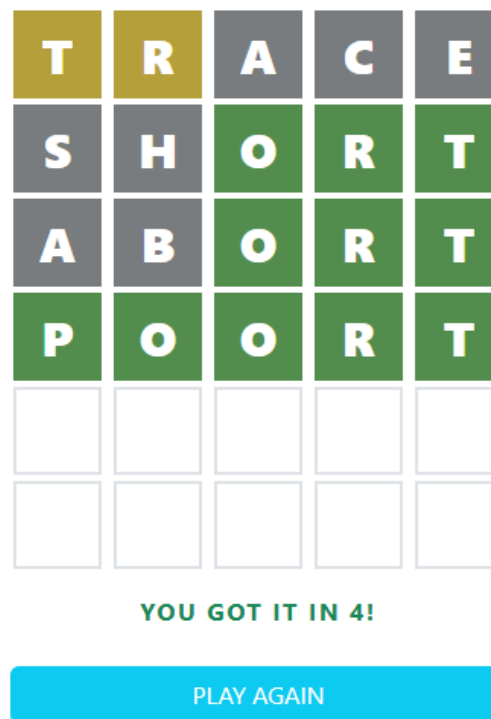


Figure 3.1 screenshot for feature 2 showing the user's previous guess and visual feedback for a game

Feature 3: Player's Dashboard

Figures 4.1 and 4.2 show the player's dashboard. Figure 4.1 shows the distribution of win rate and number of guesses for winning games of all the games played. Figure 4.2 shows all prior plays, including the date, the word, the win/lost status, and the number of attempts. The player can filter history based on week/month/year or all time. The statistics are computed by tracking all the completed GameSessions associated with a user.

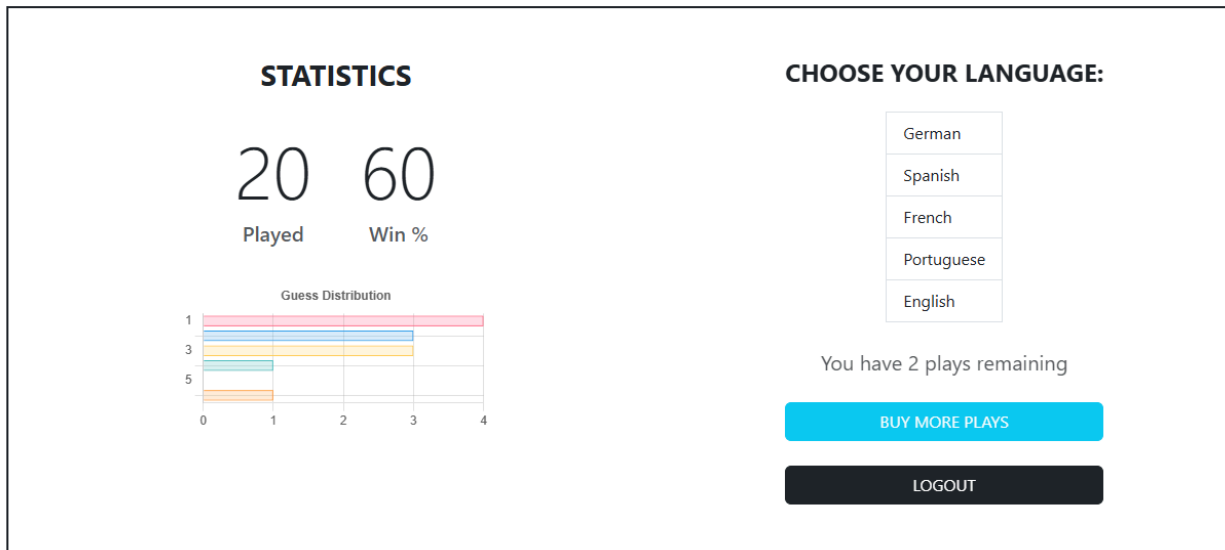


Figure 4.1 screenshot for feature 3.2 showing player's overall basic statistics

Game History

SHOW: [LAST WEEK](#) [LAST MONTH](#) [LAST YEAR](#) [ALL TIME](#)

MAY 01, 2026 [POORT](#) [Won](#) ATTEMPTS: 4

APR 30, 2026 [STOUP](#) [Lost](#) ATTEMPTS: 6

APR 30, 2026 [STROW](#) [Lost](#) ATTEMPTS: 6

APR 30, 2026 [ZANTE](#) [Won](#) ATTEMPTS: 1

APR 30, 2026 [NEPIT](#) [Won](#) ATTEMPTS: 3

APR 30, 2026 [SOLAN](#) [Won](#) ATTEMPTS: 2

APR 30, 2026 [LANAS](#) [Won](#) ATTEMPTS: 3

Figure 4.2 screenshot for feature 3.1 showing all prior plays

Feature 3: Buying More Games

Figure 5.1 and Figure 5.2 show Feature 4: Buying Game Plays. Figure 5.1 shows the current user's balance and the balance board, which includes all user's records in the database. Figure 5.2 shows the checkout process, where the user specifies how many games they'd like to purchase and gets charged accordingly.

YOUR BALANCE	
2 TOKENS	

BALANCE BOARD	
PLAYER	TOKENS
jdasilv2@nd.edu	10
cta@nd.edu	2
bmarti32@nd.edu	24
jalen@nd.edu	20
john@nd.edu	20
joe@nd.edu	100
jerry@nd.edu	20
mario@nd.edu	10
sam@nd.edu	15
boris@nd.edu	4
berry@nd.edu	19

Figure 5.1 screenshot for feature 4 showing the user's current balance and all users' balance

BUY PLAYS
1 token = 1 extra game play.
HOW MANY PLAYS?
1
BUY PLAYS

Figure 5.2 screenshot for feature 4 showing the checkout process for buying more games

Project's Learned Lessons

1. We chose a procedural programming paradigm because it is easy to read and follow. Most of the logic runs in a straight timeline, and we wanted functions to do one thing and return a result. Since they are separate, we can test them easier in isolation since they take inputs and return outputs without any states. In addition, there is less overhead compared to OOP because there is no need for design class hierarchies.

If we were to start the project from scratch, we would still choose a procedural programming paradigm since it is easier to track and debug, given the small scale of the project. Django's function-based views fit naturally with our chosen procedural programming. However, if we were to continue the project and as the project grows, `view.py` will become a large file where everything lives, and it gets hard to navigate. Some of the functions are loosely connected, so there is no notion of relationship. While we can reuse functions, we do end up passing the same objects into multiple functions, making overall code reuse harder.

2. One of the most challenging aspects is deciding where to redirect requests and logic to which view. We have 5 different views to handle redirecting users throughout the game. It is difficult to keep track of all the views' functions and decide where the authentication logic should go. For example, which view should verify that the player has enough play remaining? Since the user can go to many different pages, it was hard to keep track of which view the user is in or which app we should redirect the player to.

Another challenge is handling different languages inside the HTML template. For example, we use Python via Django to send the necessary information to render a page. Inside the page, we use JS to get the data from Django and handle the interactiveness with the web page. We also use a combination of Bootstrap and our own CSS, which added a lot of complexities to the page. We had to switch between different languages and different syntaxes, and make sure they work together.

3. The largest positive impact on our project's outcome comes from our organization of tasks. We divided up our work by features, which allows us to work independently and efficiently. It happens that our features end up as separate apps, which makes it easier to isolate and debug a specific feature. In addition, since each is a different app, when we merge different features, we only have to add URL redirections.